

Git Tutorial for Web Programming Students

Sep 30, 2026 · @Here is the code for hand fingers detection and tracking using g

What Is Git and Why Use It?

Git is a **version control system**: it saves a history of every change you make to your code, so you can always go back, compare versions, and share your work safely.

Think of it like Google Docs for code — except it saves only when *you* decide to save a checkpoint (called a **commit**).

Without Git:

- You accidentally delete something → it's gone
- You want to try a new idea but are afraid to break working code
- You can't easily share code with teammates or professors

With Git:

- Every version is saved forever — you can always go back
- You can experiment in a branch without touching your working code
- Your code lives on GitHub — shareable with one URL, hosted for free

Git vs. GitHub — what's the difference?

	Git	GitHub
What it is	A tool installed on your computer	A website that stores your Git repos
What it does	Tracks changes locally	Shares and backs up your repo online
Analogy	Like saving a file	Like uploading to Google Drive

You use *Git* on your computer, and *push* to *GitHub* to share online.

Setup: Install Git and Configure It

Step 1: Download and install Git

1. Go to git-scm.com/downloads

2. Download for your OS (Windows / Mac / Linux)
3. Install with **default settings** (click Next through all screens)

Step 2: Verify the installation

Open a terminal:

- **Windows:** search for **Git Bash** in the Start menu
- **Mac/Linux:** open **Terminal**

```
git --version  
# Expected output: git version 2.x.x
```

Step 3: Tell Git who you are (do this once)

Git stamps every commit with your name and email. Use the same email as your GitHub account.

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"  
  
# Verify it was saved  
git config --global --list  
# user.name=Your Name  
# user.email=you@example.com
```

Step 4: Set VS Code as Git's default editor (recommended)

```
git config --global core.editor "code --wait"
```

Windows users: During Git installation, choose **"Git from the command line and also from 3rd-party software"** when prompted. This lets you use `git` in any terminal, not only Git Bash.

Core Concepts

Four ideas explain 90% of how Git works.



The four zones explained

Working directory — your actual files, open in VS Code. Changes here are not saved to Git yet.

Staging area — a "waiting room" where you place files you want to include in the next commit. `git add` moves files here.

Local repo — stored inside the hidden `.git/` folder in your project. `git commit` saves a permanent snapshot here.

Remote repo — the copy of your repo on GitHub. `git push` sends your commits there; `git pull` brings down changes from there.

Commits — saved snapshots

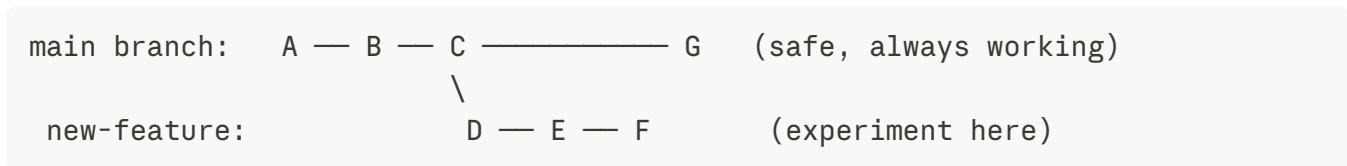
A commit is a snapshot of your project at one moment in time. Every commit has:

- A unique ID (called a hash): `a3f92c1`
- Your name and email
- A message describing what changed
- A timestamp

You can always go back to any commit and see exactly what the code looked like.

Branches — parallel versions

A branch lets you work on something new without touching the main code.



When the feature is ready, you **merge** it back into `main`. For this class, you will mostly work on `main` directly.

The Daily Workflow

Every time you write code, follow this 3-command cycle:

```
Edit files → git add → git commit → git push
```

Step-by-step with full commands

```
# 1. Check what has changed since your last commit
git status
# Shows: modified files (red), staged files (green), untracked new files

# 2. Stage the files you want to commit
git add index.html          # add one specific file
git add week1/              # add an entire folder
git add .                   # add EVERYTHING that changed

# 3. Save a snapshot (commit)
git commit -m "Add Week 1 HTML basics page"
# The message should describe WHAT changed, in plain English

# 4. Upload to GitHub
git push
```

What does git status tell you?

```
git status

# Example output:
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
       modified:   index.html          <-- changed, not yet staged

Untracked files:
  (use "git add <file>..." to include in what will be committed)
       week2/css-basics.html          <-- new file, not yet tracked
```

Red = not staged (Git knows about it but won't include it in the next commit) **Green = staged** (will be included in the next commit)

Writing good commit messages

```
# Good: specific and clear
git commit -m "Add counter app with add/subtract buttons"
git commit -m "Fix broken link to week2 folder"
git commit -m "Update CSS colors for dark background"

# Bad: vague
git commit -m "update"
git commit -m "fix stuff"
git commit -m "aaa"
```

Seeing your history

```
git log --oneline
# Shows all commits, newest first:
# a3f92c1 Add counter app with add/subtract buttons
# b7e4d2a Fix broken link to week2 folder
# c1d83f5 Initial commit
```

Going back to a previous version

```
# View what a file looked like at a specific commit
git show a3f92c1:index.html

# Discard ALL your unsaved changes (careful – cannot undo)
git restore index.html
```

GitHub: Connect Your Repo

Create the repo on GitHub

1. Log in to github.com
2. Click + → **New repository**
3. Name: `webpro` → **Public** → **do NOT** check "Add a README" (we'll push one from your computer)

4. Click **Create repository**

5. Copy the repo URL: `https://github.com/USERNAME/webpro.git`

First push from your computer

Run these commands once per new project:

```
# Start tracking your folder with Git
git init

# Create an initial file (if you haven't already)
echo "# My Web Portfolio" > README.md

# Stage and commit
git add .
git commit -m "Initial commit"

# Tell Git where GitHub is
git remote add origin https://github.com/USERNAME/webpro.git

# Push your code to GitHub
git branch -M main
git push -u origin main
```

After this first push, your daily workflow is just the 3 commands: `git add` → `git commit` → `git push`.

Clone an existing repo (alternative: start from GitHub)

If you created the repo on GitHub first (recommended for beginners):

```
git clone https://github.com/USERNAME/webpro.git
cd webpro
# Now edit files, then:
git add .
git commit -m "My first change"
git push
```

Authentication

GitHub no longer accepts passwords on the command line. Use one of these:

- **HTTPS + personal access token:** Settings → Developer settings → Personal access tokens → Generate new token (scope: `repo`). Use this token instead of your password when prompted.
- **GitHub Desktop** (beginner-friendly): desktop.github.com — a visual app that handles authentication for you.

GitHub Pages: Deploy Your Site

GitHub Pages turns your repo into a live website for free.

Enable GitHub Pages

1. Open your repo on GitHub
2. Click **Settings** (top tab) → **Pages** (left sidebar)
3. Under **Source:** select **Deploy from a branch**
4. Branch: **main** / folder: **/ (root)** → click **Save**
5. Wait 1-2 minutes → refresh — your URL appears:

```
https://USERNAME.github.io/webpro/
```

How Pages decides what to show

What you visit	What GitHub Pages serves
<code>username.github.io/webpro/</code>	<code>index.html</code> in the root
<code>username.github.io/webpro/week1/</code>	<code>week1/index.html</code>
<code>username.github.io/webpro/week1/forms.html</code>	<code>week1/forms.html</code>

Update your live site

You do not need to do anything special. Every time you `git push` , GitHub Pages rebuilds your site automatically within 1-2 minutes.

```
# Edit a file, then:
git add .
git commit -m "Update week2 CSS"
git push
# Site is live again in ~1 minute
```

Check if your site built successfully

If the site doesn't update, go to: **GitHub repo → Actions tab** — you will see a yellow spinner (building) or a green check (done) or a red X (error — click it to see what went wrong).

Common Mistakes and Fixes

“my push was rejected”

```
# Error message:
# error: failed to push some refs to 'https://github.com/...'
# hint: Updates were rejected because the remote contains work that you do
# not have locally.

# Fix: pull first, then push
git pull --rebase
git push
```

This happens when someone else (or you from another device) pushed commits you don't have yet.

“my site is not updating on GitHub Pages”

1. Wait 2 more minutes — Pages is sometimes slow
 2. Hard-refresh the browser: `Ctrl + Shift + R` (Windows) or `Cmd + Shift + R` (Mac)
 3. Check: **Repo → Actions tab** — is there a red X? Click it for the error details
 4. Make sure you pushed: `git log --oneline` — do you see your last commit?
-

“I committed the wrong file / with a typo in the message”

```
# Fix the last commit message (BEFORE pushing)
git commit --amend -m "Correct message here"

# Undo the last commit but KEEP your file changes (BEFORE pushing)
git reset HEAD~1
```

Never use `--amend` or `reset` on commits you have already pushed — it rewrites history and causes problems for anyone else using the repo.

“git says there are conflicts”

```
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
```

Open the file. Git marks the conflict like this:

```
<<<<<<< HEAD
<h1>My title (your version)</h1>
=====
<h1>My title (other version)</h1>
>>>>>>> origin/main
```

1. Delete the `<<<<<<<`, `=====`, `>>>>>>>` lines
2. Keep the version you want (or combine both)
3. Save, then:

```
git add index.html
git commit -m "Resolve merge conflict"
git push
```

“I deleted a file by accident”

```
# Restore a deleted file from the last commit
git restore index.html
```

“git push asks for a password but rejects it”

GitHub removed password authentication in 2021. You need a **Personal Access Token**:

1. GitHub → Profile → Settings → Developer settings → Personal access tokens → Tokens (classic)
2. Generate new token → scope: check `repo` → copy the token
3. Use the token as your password when Git asks

Quick Reference Card

Print or bookmark this. These commands cover 95% of day-to-day Git use.

Setup (once per computer)

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Start a project

```
git init                # Start tracking a folder
git clone https://github.com/...  # Download an existing repo
```

Daily workflow

```
git status              # What changed?
git add .               # Stage everything
git add filename.html   # Stage one file
git commit -m "Describe what changed" # Save a snapshot
git push               # Upload to GitHub
git pull               # Download latest from GitHub
```

History and comparison

```
git log --oneline       # List all commits
git diff               # Show unstaged changes
git show a3f92c1       # Show a specific commit
```

Undo (use carefully)

```
git restore filename.html      # Discard unsaved changes to a file
git reset HEAD~1               # Undo last commit (keep changes)
git commit --amend -m "New message" # Fix last commit message (not pushed yet)
```

Branches (intermediate)

```
git branch feature-quiz      # Create a new branch
git checkout feature-quiz    # Switch to it
git checkout main             # Switch back to main
git merge feature-quiz       # Merge a branch into current
```

The 5 most important commands for beginners

Command	What it does
<code>git status</code>	Show what changed
<code>git add .</code>	Stage all changes
<code>git commit -m "..."</code>	Save a snapshot
<code>git push</code>	Upload to GitHub
<code>git pull</code>	Download from GitHub

Reference video: [Git and GitHub for Beginners](#) by Fireship (13 minutes)